INFORMATIONSBLATT

IB02 · Bash für die Systemadministration

LF10b · LS03 – Administrationsaufgaben mit Bash automatisieren · IB02 – Bash für die Systemadministration | IT-3B · Schuljahr 2025/2026

1 · Bedingungen und Dateisystemprüfungen

1.1 | if-else-Struktur

Mit `if` prüfen Sie eine Bedingung und reagieren entsprechend. Der Block endet immer mit `fi` (if rückwärts).

if-else – Grundstruktur

```
if [ BEDINGUNG ]; then
    # wird ausgeführt, wenn Bedingung WAHR ist
elif [ ANDERE_BEDINGUNG ]; then
    # optionaler zweiter Zweig
else
    # wird ausgeführt, wenn keine Bedingung zutrifft
fi
```

1.2 | Häufige Datei- und Verzeichnisprüfungen

Ausdruck	Bedeutung	Beispiel
`[ -d "$PFAD" ]`	Verzeichnis existiert	Prüfen, ob `/backup` da ist
`[ ! -d "$PFAD" ]`	Verzeichnis existiert *nicht*	Anlegen, falls fehlend
`[ -f "$DATEI" ]`	Datei existiert und ist reguläre Datei	Config-Datei vorhanden?
`[ -r "$DATEI" ]`	Datei ist lesbar	Vor dem Backup prüfen
`[ -x "$DATEI" ]`	Datei ist ausführbar	Skript-Prüfung
`[ "$A" = "$B" ]`	String-Vergleich: A gleich B	Parameter prüfen
`[ $? -eq 0 ]`	Letzter Befehl war erfolgreich	Nach `tar` prüfen
`[ $? -ne 0 ]`	Letzter Befehl ist fehlgeschlagen	Fehlerbehandlung

Praxisbeispiel: Verzeichnis prüfen und anlegen

```
ZIEL="/backup/web"

if [ ! -d "$ZIEL" ]; then
    mkdir -p "$ZIEL"
    echo "Verzeichnis $ZIEL wurde angelegt."
fi
```

✅ **Tipp: Kurzform mit &&**

Wenn Sie nur einen Befehl ausführen wollen, falls eine Bedingung wahr ist, können Sie `&&` nutzen – kürzer und lesbar:

```
[ ! -d "$ZIEL" ] && mkdir -p "$ZIEL"
```

2 · Exit-Codes und Fehlerbehandlung

📖 **Exit-Code**

Jeder Linux-Befehl beendet sich mit einem numerischen **Exit-Code**. Der Wert `0` bedeutet immer Erfolg. Alle anderen Werte (1, 2, 127, …) signalisieren einen Fehler. Der Exit-Code des zuletzt ausgeführten Befehls steht in der Variable `$?`.

Exit-Codes auswerten – Backup-Beispiel

```
tar -czf "$ARCHIV" "$QUELLE"

if [ $? -eq 0 ]; then
    echo "Backup erfolgreich erstellt: $ARCHIV"
else
    echo "FEHLER: Backup fehlgeschlagen!" >&2
    exit 1
fi
```

Element	Bedeutung
`$?`	Exit-Code des letzten Befehls
`-eq 0`	Gleich 0 (Erfolg)
`-ne 0`	Ungleich 0 (Fehler)
`>&2`	Ausgabe auf Standardfehler (stderr) umleiten
`exit 1`	Skript mit Fehlercode beenden

3 · Schleifen

3.1 | for-Schleife

Eine `for`-Schleife wiederholt einen Block für jeden Wert in einer Liste.

for-Schleife – Dateien verarbeiten

```
# Alle .log-Dateien in einem Verzeichnis anzeigen
for DATEI in /var/log/apache2/*.log; do
    echo "Verarbeite: $DATEI"
    # Hier könnten weitere Aktionen folgen
done

# Schleife über eine feste Liste
for DIENST in apache2 cron ssh; do
    systemctl is-active --quiet "$DIENST" && echo "$DIENST: aktiv" || echo "$DIENST: inaktiv"
done
```

3.2 | while-Schleife

while-Schleife – Zähler

```
# Schleife solange ZAEHLER kleiner als 5 ist
ZAEHLER=1
while [ $ZAEHLER -le 5 ]; do
    echo "Durchlauf: $ZAEHLER"
    ZAEHLER=$((ZAEHLER + 1))
done
```

4 · Ausgaben und Logging

4.1 | Umleitungsoperatoren

Operator	Bedeutung	Anwendung
`>`	Ausgabe in Datei schreiben (überschreiben)	`echo "Text" > datei.log`
`>>`	Ausgabe an Datei anhängen	`echo "Text" >> datei.log`
`2>`	Fehlerausgabe (stderr) in Datei	`befehl 2> fehler.log`
`2>&1`	Fehlerausgabe auf normale Ausgabe umleiten	`befehl >> alles.log 2>&1`
`>&2`	Normale Ausgabe auf stderr umleiten	`echo "FEHLER" >&2`

💡 **Merke: 2>&1 vs. >&2 – nicht verwechseln!**

- `2>&1` – Fehlerkanal (2) wird auf die normale Ausgabe (1) umgeleitet → beide Ausgaben landen in derselben Logdatei
- `>&2` – Normale Ausgabe wird auf den Fehlerkanal umgeleitet → Fehlermeldungen im Skript mit `echo "FEHLER" >&2` korrekt kennzeichnen

4.2 | Logging-Funktion

Statt überall `echo "$(date) - ..."` zu schreiben, definieren Sie einmalig eine **Funktion**. Eine Funktion ist ein benannter Codeblock, der beliebig oft aufgerufen werden kann.

Wiederverwendbare Logging-Funktion

```
LOGDATEI="/var/log/netconsult/backup.log"

# Funktion definieren
log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$LOGDATEI"
}

# Funktion aufrufen – $1 ist der übergebene Text
log "Backup gestartet."
log "Backup erfolgreich: $ARCHIV"
log "FEHLER: Backup fehlgeschlagen."
```

⚠️ Logverzeichnis gehört root
Die Logdatei liegt unter `/var/log/netconsult/`. Das Verzeichnis `/var/log` gehört **root** – ein normaler Benutzer kann `/var/log/netconsult` weder anlegen noch hineinschreiben. Das Skript legt das Verzeichnis daher selbst an (`mkdir -p`) und läuft mit **root-Rechten** (über die root-Crontab; zum Testen mit `sudo`). Ohne root scheitert es mit *„Permission denied"*.

Beispiel-Ausgabe in der Logdatei:

/var/log/netconsult/backup.log

```
2025-09-15 03:00:01 - Backup gestartet.
2025-09-15 03:00:04 - Backup erfolgreich: /backup/web/web_backup_2025-09-15_03-00.tar.gz
2025-09-15 03:00:04 - Alte Backups gelöscht: 2 Dateien entfernt.
```

5 · Dateiarchivierung mit tar

Befehl	Bedeutung
`tar -czf archiv.tar.gz /pfad/`	Komprimiertes Archiv erstellen (gzip)
`tar -tzf archiv.tar.gz`	Inhalt des Archivs anzeigen (ohne Entpacken)
`tar -xzf archiv.tar.gz -C /ziel/`	Archiv entpacken nach /ziel/

Bedeutung der Optionen: `c` = create, `z` = gzip, `f` = Dateiname angeben, `x` = extract, `t` = list

tar im Skript – Backup mit Datumsstempel

```
QUELLE="/var/www/html/netconsult"
ZIEL="/backup/web"
DATUM=$(date +%Y-%m-%d_%H-%M)
ARCHIV="$ZIEL/web_backup_$DATUM.tar.gz"

tar -czf "$ARCHIV" "$QUELLE"

if [ $? -eq 0 ]; then
    log "Backup erfolgreich: $ARCHIV"
else
    log "FEHLER: tar ist fehlgeschlagen!"
    exit 1
fi
```

6 · Suche mit find

find – Dateien nach Alter suchen und löschen

```
# Dateien älter als 7 Tage im Verzeichnis /backup/web anzeigen
find /backup/web -name "*.tar.gz" -mtime +7

# Diese Dateien direkt löschen
find /backup/web -name "*.tar.gz" -mtime +7 -delete

# Dateien älter als 30 Tage in /var/log/apache2 anzeigen
find /var/log/apache2 -type f -mtime +30
```

Option	Bedeutung
`-name "*.tar.gz"`	Nur Dateien mit diesem Namensmuster
`-type f`	Nur reguläre Dateien (nicht Verzeichnisse)
`-mtime +7`	Zuletzt geändert vor mehr als 7 Tagen
`-delete`	Gefundene Dateien löschen

> ⚠️ **Vorsicht mit -delete**
> Führen Sie `find` immer erst *ohne* `-delete` aus, um zu prüfen, welche Dateien gefunden werden. Erst wenn das Ergebnis stimmt, fügen Sie `-delete` hinzu.

7 · systemctl in Skripten verwenden

Der Befehl `systemctl is-active --quiet DIENST` ist die skript-optimierte Variante: Er erzeugt **keine Ausgabe**, gibt aber den Exit-Code zurück (0 = aktiv, ungleich 0 – meist 3 – = nicht aktiv). Damit lässt er sich direkt in einer if-Bedingung nutzen.

Befehl	Verwendung in Skripten
`systemctl is-active --quiet apache2`	Direkt in if-Bedingung – keine störende Ausgabe
`systemctl restart apache2`	Dienst neu starten (z. B. nach erkanntem Ausfall)
`systemctl enable apache2`	Dienst für Autostart aktivieren
`systemctl status apache2`	Ausführlicher Status (für manuelle Prüfung)

Apache-Dienststatus in einem Skript prüfen (Ubuntu 24.04)

```
if systemctl is-active --quiet apache2; then
    log "Apache läuft – Status OK."
else
    log "WARNUNG: Apache ist nicht aktiv. Starte neu..."
    systemctl restart apache2
    if [ $? -eq 0 ]; then
        log "Apache erfolgreich neugestartet."
    else
        log "FEHLER: Neustart fehlgeschlagen!"
        exit 1
    fi
fi
```

LF10b – Serverdienste bereitstellen und Administrationsaufgaben automatisieren · LS03 – Administrationsaufgaben mit Bash automatisieren · IB02 – Bash für die · Schuljahr 2025/2026